

Simple linear regression

Throughout the following exercises, we will learn to use Python to build a simple linear regression model. Before starting on this programming exercise, we strongly recommend watching the recommended video lessons

All the information you need for solving this assignment is in this notebook, and all the code you will be implementing will take place within this notebook.

Setup: libraries we'll use

As we move forward, you can find instructions on how to install required libraries as they arise in this notebook. Before we begin the exercises and analyzing the data, we need to import all libraries required for this programming exercise.

We will be using:

- **pandas** for data manipulation
- **statsmodels** for statistical modeling
- **seaborn** for plotting

Why statsmodels?

Statsmodels is built for **statistical inference**, not just prediction. It fits models like OLS, GLM, ARIMA, and logit the way you would in R or Stata, and returns a full summary table with coefficients, standard errors, p-values, confidence intervals, R², AIC, and diagnostic tests.

That makes it ideal when the question is "is this effect real, and how large is it?" rather than "what will this model predict next?"

What are the alternatives, and how are they different?

- **scikit-learn**: the workhorse for machine learning in Python. It focuses on predictive modeling with a consistent `.fit()` / `.predict()` API, pipelines, cross-validation, and regularization. You get excellent out-of-sample accuracy, but by default you do *not* get p-values or confidence intervals. Use scikit-learn when deployment, automation, and prediction matter more than hypothesis testing.
- **linearmodels**: extends the statsmodels philosophy to panel data. If you later need fixed-effects, random-effects, or instrumental-variable regressions, this is the natural next step.
- **arch**: specializes in volatility models (GARCH family) for financial time series, where statsmodels' basic ARIMA is not enough.
- **PyMC / Bambi**: Bayesian alternatives. Instead of point estimates and p-values, they return full posterior distributions. Use them when you want to incorporate priors or quantify uncertainty more richly.
- **scipy.stats**: lightweight and great for one-off tests (t-test, chi-square, correlation), but it does not fit multivariate models.

In short: we choose statsmodels in this stage because we are teaching explanation and inference first. You will see scikit-learn later when we shift from "what drives y?" to "how well can we predict y?" The two libraries often coexist — statsmodels to understand, scikit-learn to deploy.

Relevant imports

Begin by importing the relevant packages and data.

```
# Import packages
import pandas as pd
import numpy as np
import seaborn as sns
```

Note: Recall that the default for `head()` is to show the first 5 rows. If you change the value for `(n)`, you can show more rows. For example, if you load the sns dataset and call it "penguins," the command `penguins.head(3)` will show 3 rows.

```
# Load dataset
penguins = sns.load_dataset("penguins")

# Examine first 5 rows of dataset
penguins.head(10)
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female
5	Adelie	Torgersen	39.3	20.6	190.0	3650.0	Male
6	Adelie	Torgersen	38.9	17.8	181.0	3625.0	Female
7	Adelie	Torgersen	39.2	19.6	195.0	4675.0	Male
8	Adelie	Torgersen	34.1	18.1	193.0	3475.0	NaN
9	Adelie	Torgersen	42.0	20.2	190.0	4250.0	NaN

From the first 5 rows of the dataset, we can see that there are several columns available: `species`, `island`, `bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`, `body_mass_g`, and `sex`. There also appears to be some missing data.

Data cleaning

For the purposes of this lecture, we are focusing our analysis on Adelie and Gentoo penguins, and will be dropping any missing values from the dataset. In a work setting, you would typically examine the data more thoroughly before deciding how to handle missing data (i.e., fill in, drop, etc.).

```
penguins['species'].head()
```

	species
0	Adelie
1	Adelie
2	Adelie
3	Adelie
4	Adelie

dtype: object

```
# Keep Adelie and Gentoo penguins, drop missing values
penguins_sub = penguins[penguins["species"] != "Chinstrap"]
penguins_final = penguins_sub.dropna()
penguins_final.reset_index(inplace=True, drop=True)
```

You can review the documentation for `dropna()` and `reset_index()`. In short, the `dropna()` function by default removes any rows with any missing values in any of the columns. The `reset_index()` function resets the index values for the rows in the dataframe. Typically, you use `reset_index()` after you've finished manipulating the dataset. By setting `inplace=True`, you will not create a new DataFrame object. By setting `drop=True`, you will not insert a new index column into the DataFrame object.

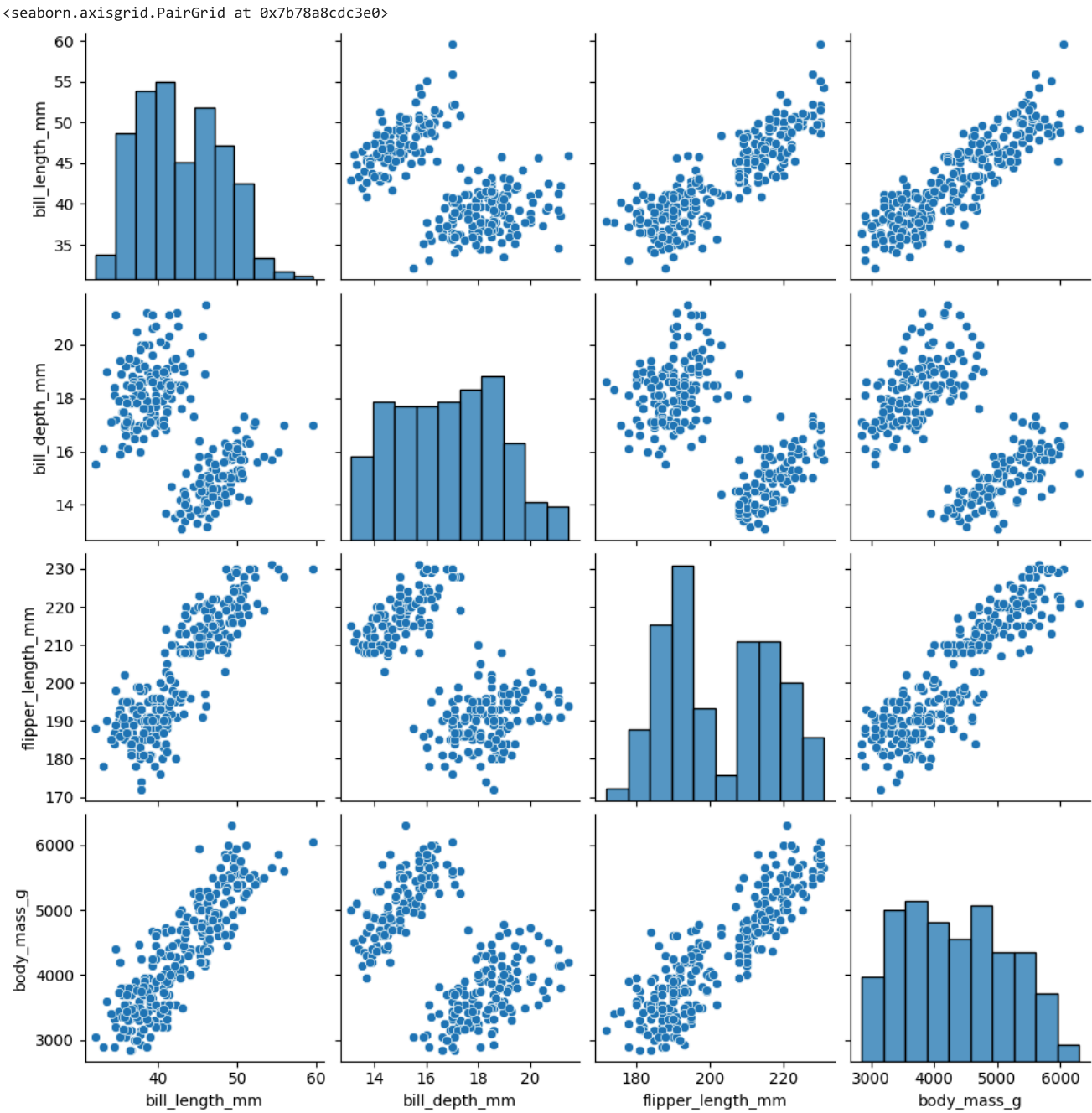
Exploratory data analysis

Before you construct any model, it is important to get more familiar with your data. You can do so by performing exploratory data analysis or EDA.

Since this part of the lecture focuses on simple linear regression, you want to check for any linear relationships among variables in the dataframe. You can do this by creating scatterplots using any data visualization package, for example `matplotlib.pyplot`, `seaborn`, or `plotly`.

To visualize more than one relationship at the same time, we use the `pairplot()` function from the `seaborn` package to create a scatterplot matrix.

```
# Create pairwise scatterplots of data set
sns.pairplot(penguins_final)
```



From the scatterplot matrix, you can observe a few linear relationships:

- bill length (mm) and flipper length (mm)
- bill length (mm) and body mass (g)
- flipper length (mm) and body mass (g)

Model construction

Based on the above scatterplots, you could probably run a simple linear regression on any of the three relationships identified. For this part of the lecture, we will focus on the relationship between bill length (mm) and body mass (g).

To do this, you will first subset the variables of interest from the dataframe. You can do this by using double square brackets `[[]]`, and listing the names of the columns of interest.

```
# Subset Data

ols_data = penguins_final[["body_mass_g", "bill_length_mm", ]]
```

Next, you can construct the linear regression formula, and save it as a string. Remember that the y or dependent variable comes before the `~`, and the x or independent variables comes after the `~`.

Note: The names of the x and y variables have to exactly match the column names in the dataframe.

```
# Write out formula
ols_formula = "body_mass_g ~ bill_length_mm"
```

$y = bx + c + e$

Lastly, you can build the simple linear regression model in `statsmodels` using the `ols()` function. You can import the `ols()` function directly using the line of code below.

```
# Import ols function
from statsmodels.formula.api import ols
```

Then, you can plug in the `ols_formula` and `ols_data` as arguments in the `ols()` function. After you save the results as a variable, you can call on the `fit()` function to actually fit the model to the data.

```
# Build OLS, fit model to data
OLS = ols(ols_formula, ols_data)
model = OLS.fit()
```

Lastly, you can call the `summary()` function on the `model` object to get the coefficients and more statistics about the model. The output from `model.summary()` can be used to evaluate the model and interpret the results. Later in this section, we will go over how to read the results of the model output.

```
model.summary()
```

OLS Regression Results

Dep. Variable:	body_mass_g	R-squared:	0.769
Model:	OLS	Adj. R-squared:	0.768
Method:	Least Squares	F-statistic:	874.3
Date:	Thu, 04 Jun 2026	Prob (F-statistic):	1.33e-85
Time:	20:07:56	Log-Likelihood:	-1965.8
No. Observations:	265	AIC:	3936.
Df Residuals:	263	BIC:	3943.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-1707.2919	205.640	-8.302	0.000	-2112.202	-1302.382
bill_length_mm	141.1904	4.775	29.569	0.000	131.788	150.592

Omnibus: 2.060 **Durbin-Watson:** 2.067

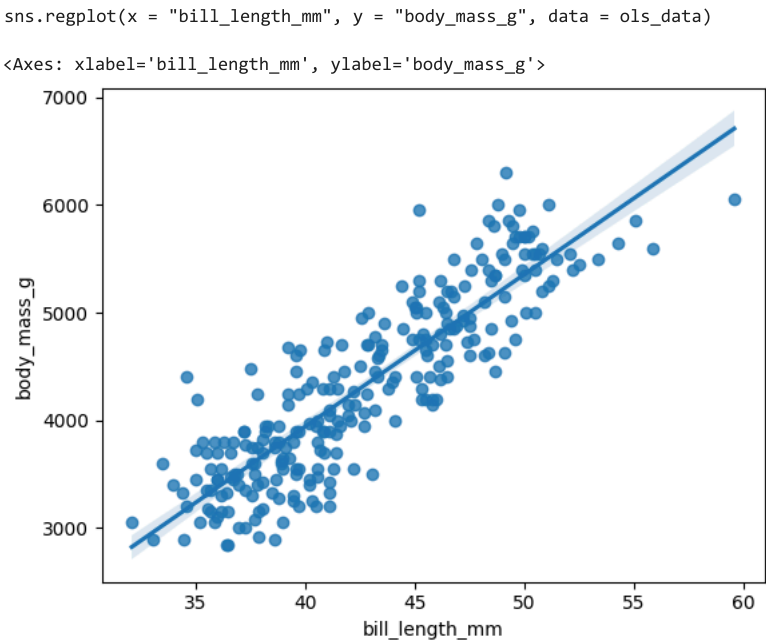
Prob(Omnibus): 0.357 **Jarque-Bera (JB):** 2.103

Skew: 0.210 **Prob(JB):** 0.349

Kurtosis: 2.882 **Cond. No.** 357.

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

You can use the `regplot()` function from `seaborn` to visualize the regression line.



Finish checking model assumptions

There are four main model assumptions for simple linear regression, in no particular order:

- 1. Linearity
- 2. Normality
- 3. Independent observations
- 4. Homoscedasticity

You already checked the linearity assumption by creating the scatterplot matrix. The independent observations assumption is more about data collection. There is no reason to believe that one penguin's body mass or bill length would be related to any other penguin's anatomical measurements. So we can check off assumptions 1 and 3.

The normality and homoscedasticity assumptions focus on the distribution of errors. Thus, you can only check these assumptions after you have constructed the model. To check these assumptions, you will check the residuals, as an approximation of the errors.

To more easily check the model assumptions and create relevant visualizations, you can first subset the X variable by isolating just the `bill_length_mm` column. Additionally, you can save the predicted values from the model using the `model.predict(X)` function.

```
# Subset X variable
X = ols_data["bill_length_mm"]

X.head()
```

	bill_length_mm
0	39.1
1	39.5
2	40.3
3	36.7
4	39.3

dtype: float64

```
# Get predictions from model
fitted_values = model.predict(X)
```

```
fitted_values.head()
```

	θ
0	3813.254231
1	3869.730406
2	3982.682755
3	3474.397182
4	3841.492318

dtype: float64

Then, you can save the model residuals as a variable by using the `model.resid` attribute.

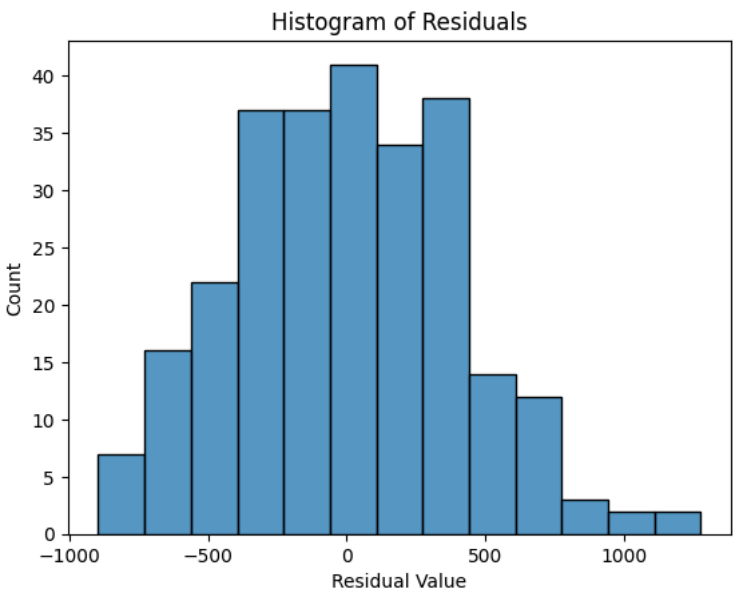
```
# Calculate residuals
residuals = model.resid
```

Check the normality assumption

To check the normality assumption, you can create a histogram of the residuals using the `histplot()` function from the `seaborn` package.

From the below histogram, you may notice that the residuals are almost normally distributed. In this case, it is likely close enough that the assumption is met.

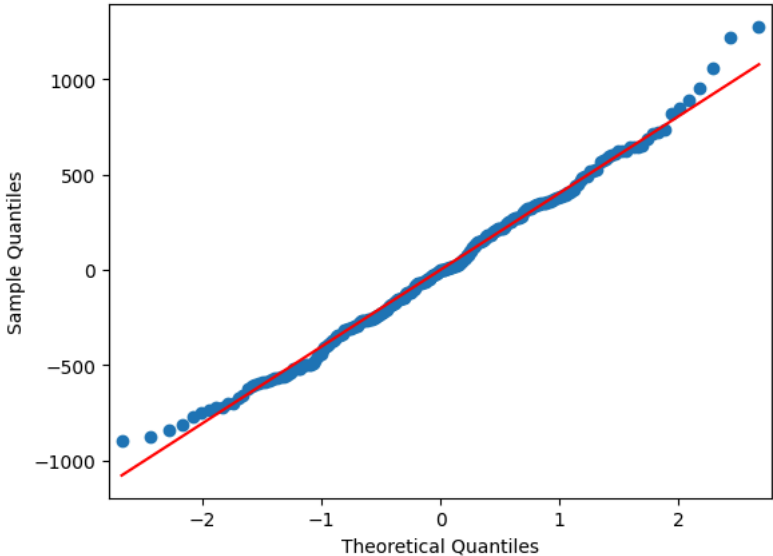
```
import matplotlib.pyplot as plt
fig = sns.histplot(residuals)
fig.set_xlabel("Residual Value")
fig.set_title("Histogram of Residuals")
plt.show()
```



Another way to check the normality function is to create a quantile-quantile or Q-Q plot. If the residuals are normally distributed, you would expect a straight diagonal line going from the bottom left to the upper right of the Q-Q plot. You can create a Q-Q plot by using the `qqplot` function from the `statsmodels.api` package.

The Q-Q plot shows a similar pattern to the histogram, where the residuals are mostly normally distributed, except at the ends of the distribution.

```
import matplotlib.pyplot as plt
import statsmodels.api as sm
fig = sm.qqplot(model.resid, line = 's')
plt.show()
```



✓ Check the homoscedasticity assumption

Lastly, we have to check the homoscedasticity assumption. To check the homoscedasticity assumption, you can create a scatterplot of the fitted values and residuals. If the plot resembles a random cloud (i.e., the residuals are scattered randomly), then the assumption is likely met.

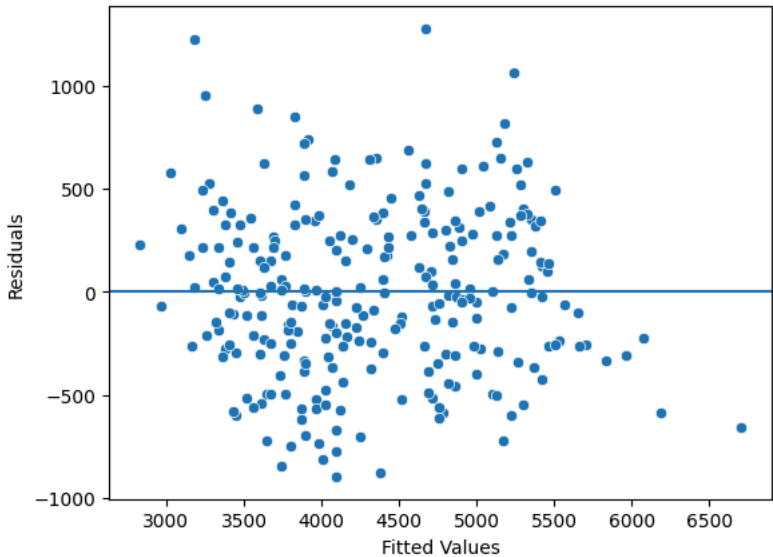
You can create one scatterplot by using the `scatterplot()` function from the `seaborn` package. The first argument is the variable that goes on the x-axis. The second argument is the variable that goes on the y-axis.

```
# Import matplotlib
import matplotlib.pyplot as plt
fig = sns.scatterplot(x=fitted_values, y=residuals)

# Add reference line at residuals = 0
fig.axhline(0)

# Set x-axis and y-axis labels
fig.set_xlabel("Fitted Values")
fig.set_ylabel("Residuals")

# Show the plot
plt.show()
```



Congratulations! You can now use start using simple linear regression models with your own datasets.

✓ Appendix

Introducing Regression Evaluation Metrics

=====

When building a machine learning model, it's essential to evaluate its performance. Evaluation metrics help us understand how well our model is doing. In this notebook, lets explore the most common evaluation metrics used in regression problems.

Mean Squared Error (MSE)

MSE measures the average squared difference between predicted and actual values. It's a popular metric for regression problems.

- **Formula:**

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_{true} - y_{pred})^2$$

- **Low MSE:** The model is making accurate predictions.

- **High MSE:** The model is making inaccurate predictions.

Mean Absolute Error (MAE)

MAE measures the average absolute difference between predicted and actual values. It's similar to MSE, but it uses absolute differences instead of squared differences.